

# Dynamic Programming for Door-Key Navigation

Edward Wang  
ECE 276B: Planning & Learning in Robotics  
University of California, San Diego  
Spring 2026

**Abstract**—This project studies optimal navigation with interaction constraints in a Door-Key grid world. The task is formulated as a finite Markov Decision Process (MDP) with non-uniform action costs, and solved using a dynamic-programming-style shortest-path method (Dijkstra over the induced state graph). The implementation is evaluated on both required settings: 7 known maps and 36 random maps. It achieves 7/7 success on known maps, 36/36 success on random maps, and additionally solves all multi-start/multi-orientation stress tests on known maps.

## I. Introduction

Navigation with contact-rich interactions appears in many robotics applications, including indoor service robots (opening doors before entering rooms), warehouse robots (moving through gated aisles), and field robots (performing action-dependent traversal). The Door-Key task captures this setting in a compact benchmark: the robot must plan motion and interaction actions (pick up key, unlock door) while minimizing a weighted energy cost.

This report presents: (i) an MDP formulation with explicit state, control, dynamics, horizon, and cost terms; (ii) a Python implementation of an optimal planner; and (iii) quantitative and visual evaluation for known-map and random-map scenarios, including additional tests with different initial positions and orientations.

High-level approach (3 steps). (1) Model: represent each Door-Key instance as a finite MDP with abstract state  $x = (p_x, p_y, d, h, m)$  and nonuniform stage cost  $\ell$ . (2) Solve: compute optimal costs and actions using dynamic programming via value iteration (Bellman backups), equivalently realized in code by shortest-path search on the induced graph from  $x_0$ . (3) Extract policies: read off a stationary greedy policy  $\pi^*(x) = \arg \min_u Q(x, u)$ ; for Part A this yields map-specific optimal policies  $\pi_k^*$  on  $X_k$ , and for Part B a single universal policy  $\pi_U^*$  expressed in the same abstract state for all random instances.

## II. Problem Statement

### A. MDP Formulation

The planning problem is modeled as a deterministic finite-horizon MDP:

- State space  $X$ :

$$x_t = (p_x, p_y, d, h, m),$$

where  $(p_x, p_y)$  is agent position,  $d \in \{0, 1, 2, 3\}$  is orientation,  $h \in \{0, 1\}$  is key possession, and  $m$  is

door-status tuple (open/locked states for each door in the map).

- Control space  $U = \{\text{MF}, \text{TL}, \text{TR}, \text{PK}, \text{UD}\}$ .
- Motion model: deterministic transition

$$x_{t+1} = f(x_t, u_t).$$

Equivalent stochastic form:

$$p_f(x'|x, u) = \begin{cases} 1, & x' = f(x, u), \\ 0, & \text{otherwise.} \end{cases}$$

- Initial state  $x_0$  (explicit): let the simulator state at load/reset be

$$x_0 = (p_x^0, p_y^0, d^0, h^0, m^0),$$

where  $(p_x^0, p_y^0) \in \mathbb{Z}^2$  is the agent grid cell,  $d^0 \in \{0, 1, 2, 3\}$  is the initial orientation index (MiniGrid convention: 0 right, 1 down, 2 left, 3 up),  $h^0 \in \{0, 1\}$  indicates whether a key is already carried, and  $m^0$  lists each door's open/locked flags in a fixed door order. Table I lists  $x_0$  for each provided known map at file load; random-map instances are listed in Tables III–IV.

- Planning horizon  $T$ : first-hitting time of goal, i.e., the first  $t$  such that  $x_t \in X_g$ , where  $X_g$  is the goal-state set.
- Stage cost  $\ell(x, u)$ :

$$\ell(x, u) = \begin{cases} 1, & u \in \{\text{TL}, \text{TR}\} \\ 3, & u = \text{MF} \\ 2, & u = \text{PK} \\ 5, & u = \text{UD} \end{cases}$$

- Terminal cost  $q(x_T)$ :

$$q(x_T) = \begin{cases} 0, & x_T \in X_g, \\ +\infty, & x_T \notin X_g. \end{cases}$$

This enforces feasible plans that terminate at goal.

The optimal policy is:

$$\pi^* = \arg \min_{\pi} \left( \sum_{t=0}^{T-1} \ell(x_t, u_t) + q(x_T) \right).$$

### B. Evaluation Scenarios

- Part A (Known Map): 7 fixed maps.
- Part B (Random Map): 36 random  $10 \times 10$  maps with fixed structural constraints from the assignment.

---

**Algorithm 1** Value Iteration (tabular Bellman backup)

---

```

1: Initialize  $V(x) \leftarrow 0$  for  $x \in X_g$ , else  $V(x) \leftarrow V_{\max}$ 
2: repeat
3:    $V_{\text{old}} \leftarrow V$ 
4:   for  $x \in X \setminus X_g$  do
5:      $V(x) \leftarrow \min_{u \in U} \{ \ell(x, u) + V_{\text{old}}(f(x, u)) \}$ 
6:   end for
7: until  $\max_{x \in X} |V(x) - V_{\text{old}}(x)| < \epsilon$ 
8: return  $V$ 

```

---

### III. Technical Approach

#### A. Value Iteration (DoorKey MDP)

Value iteration is the canonical dynamic-programming method for finite MDPs. With deterministic transitions  $x' = f(x, u)$  and nonnegative costs  $\ell(x, u)$ , define Q-factors and the Bellman optimality backup

$$Q(x, u) = \ell(x, u) + V(f(x, u)), \quad (1)$$

$$V(x) = \min_{u \in U} Q(x, u), \quad x \notin X_g, \quad (2)$$

with absorbing goal set  $X_g$  and boundary values  $V(x) = 0$  for  $x \in X_g$ . Starting from  $V^{(0)}(x) = 0$  on  $X_g$  and  $V^{(0)}(x) = +\infty$  (or a sufficiently large constant) elsewhere, iterate until  $\max_x |V^{(t+1)}(x) - V^{(t)}(x)| < \epsilon$  over the finite state space.

The optimal stationary policy is the greedy policy

$$\pi^*(x) \in \arg \min_{u \in U} \{ \ell(x, u) + V(f(x, u)) \}. \quad (3)$$

#### B. Optimal policy for Known Map

For each fixed map  $k \in \{1, \dots, 7\}$ , transitions are  $f_k(x, u)$  (walls/doors differ by map). Value iteration yields  $V_k^*(x)$  on  $X_k$ , and the known-map optimal policy is

$$\pi_k^*(x) \in \arg \min_{u \in U} \{ \ell(x, u) + V_k^*(f_k(x, u)) \}, \quad x \in X_k. \quad (4)$$

In words: once  $V_k^*$  is known, the robot executes the control that minimizes one-step cost plus optimal tail cost.

#### C. Universal policy for Random Map

Across the 36 random instances, the abstract state definition is unchanged; only the underlying occupancy and door/key/goal placement differ, all accessed through the same simulator interface. Let  $f_{\text{rand}}(x, u)$  denote the transition when the current random instance is loaded. The universal (random-map) optimal policy is

$$\pi_U^*(x) \in \arg \min_{u \in U} \{ \ell(x, u) + V_{\text{rand}}^*(f_{\text{rand}}(x, u)) \}, \quad (5)$$

a single mapping from abstract states to controls; different maps induce different reachable subsets of  $X$  but the same policy representation.

#### D. Python implementation (equivalent shortest-path form)

The code applies the same optimality principle using Dijkstra's algorithm on the implicit graph generated from  $x_0$ : each edge corresponds to one control  $u$  with weight  $\ell(x, u)$  and head  $f(x, u)$ . For nonnegative  $\ell$ , Dijkstra from  $x_0$  yields the same optimal cumulative cost to  $X_g$  as backward value iteration restricted to states reachable from  $x_0$ , and greedy backtracking along parent pointers recovers an optimal action sequence consistent with (3).

- 1) Encode nodes as  $(p_x, p_y, d, h, m)$ .
- 2) Expand all five controls using the environment dynamics  $f(\cdot, u)$ .
- 3) Relax edges with weight  $\ell(x, u)$  using a min-priority queue.
- 4) Stop when a goal node is popped; backtrack to recover the optimal action sequence.

### IV. Results

#### A. Part A: Known Maps

Table II summarizes all known-map results. The planner successfully solves all 7 maps.

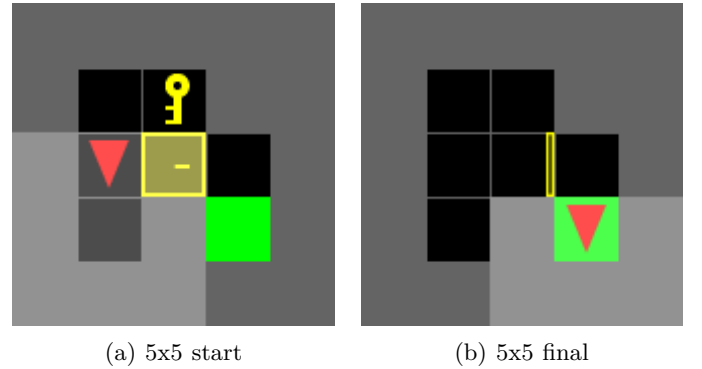


Fig. 1: Representative trajectory visualization (known map). The final frame shows the agent reaching the goal.

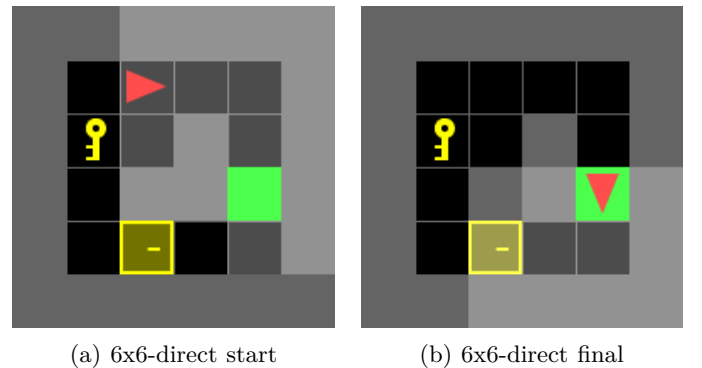


Fig. 2: Known map with short direct route.

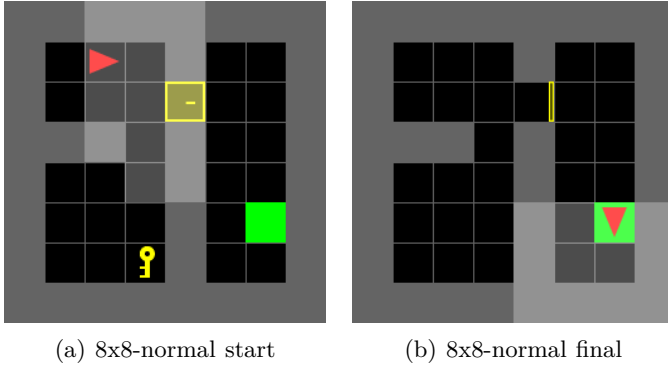


Fig. 3: Known map with longer sequence and higher total cost.

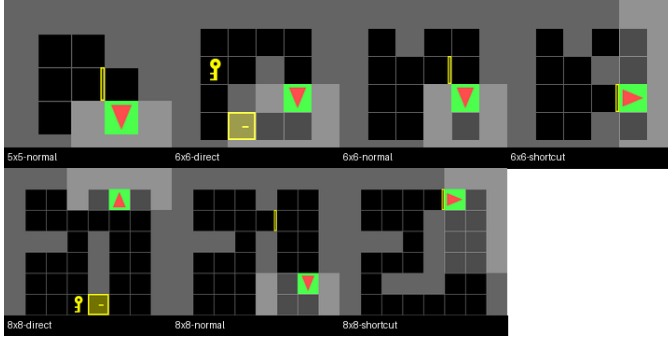


Fig. 4: Final-frame collage across all known maps. Each tile corresponds to one environment after executing the computed optimal sequence.

## B. Part B: Random Maps

The planner solves all 36 random environments:

- Success rate: 36/36
- Average steps: 11.47
- Average total cost: 30.17
- Average planning time: 0.388 s

Tables III and IV list every random-map trajectory’s total cost, together with the initial position  $(p_x^0, p_y^0)$  and orientation  $d^0$  at environment load (all instances here spawn at (4, 8) with  $d^0=3$ , i.e., facing up, per the assignment specification).

Figures 7–12 show six random-map rollouts (start vs. final frame), satisfying the requirement for multiple random-map visualizations.

To satisfy the requirement on different initial conditions, additional known-map stress tests were run with 2 initial positions and 4 initial orientations per map (8 trials/map, 56 trials total). Table V reports summary statistics; all trials succeed.

These results indicate robust feasibility and efficient planning under assignment constraints, with stable behavior across map families and initial condition changes.

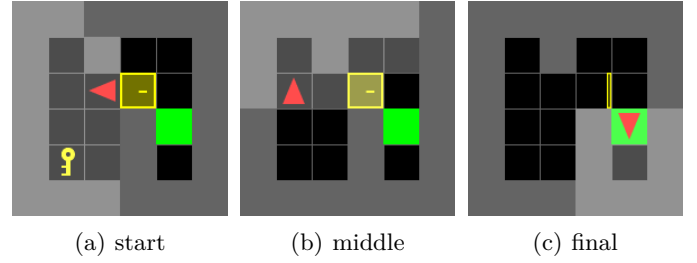


Fig. 5: Trajectory progression example for doorkey-6x6-normal.

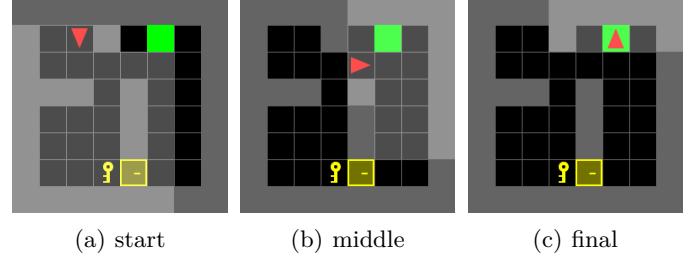


Fig. 6: Trajectory progression example for doorkey-8x8-direct.

## V. Discussion

What worked: (i) MDP state abstraction is sufficient for exact planning; (ii) positive weighted graph search gives optimal trajectories efficiently; (iii) one implementation generalizes to both known and random map settings. What did not work initially: Environment serialization and boundary edge cases in random maps caused rollout failures before adding compatibility handling in the execution wrapper.

Why: The planner assumes consistent environment semantics across files, but serialized environments can differ in internal details; robust transition handling is required for reproducible results.

Performance trends: Direct maps have lower cost/length; normal maps with longer detours and required interactions (key/door operations) show higher costs. This matches the action cost model where forward and unlock actions dominate total objective.

## VI. Conclusion

This project demonstrates that a dynamic-programming-style graph search is effective for Door-Key planning with interaction constraints. The implemented planner satisfies the required known-map and random-map tasks with 100% success on provided test sets while maintaining practical runtime.

## Acknowledgment

I followed the course policy for independent work and used only my own implementation and analysis for this submission.

TABLE I: Known maps: initial pose  $x_0$  (default spawn) and optimal trajectory cost

| Map                  | $p_x^0$ | $p_y^0$ | $d^0$ | $h^0$ | Total cost |
|----------------------|---------|---------|-------|-------|------------|
| doorkey-5x5-normal   | 1       | 2       | 1     | 0     | 28         |
| doorkey-6x6-direct   | 2       | 1       | 0     | 0     | 13         |
| doorkey-6x6-normal   | 2       | 2       | 2     | 0     | 33         |
| doorkey-6x6-shortcut | 2       | 3       | 2     | 0     | 15         |
| doorkey-8x8-direct   | 2       | 1       | 1     | 0     | 17         |
| doorkey-8x8-normal   | 2       | 1       | 0     | 0     | 54         |
| doorkey-8x8-shortcut | 2       | 1       | 3     | 0     | 19         |

TABLE II: Part A: optimal trajectory length and success (default  $x_0$ )

| Environment          | Success | Steps | Total cost |
|----------------------|---------|-------|------------|
| doorkey-5x5-normal   | True    | 13    | 28         |
| doorkey-6x6-direct   | True    | 5     | 13         |
| doorkey-6x6-normal   | True    | 14    | 33         |
| doorkey-6x6-shortcut | True    | 6     | 15         |
| doorkey-8x8-direct   | True    | 7     | 17         |
| doorkey-8x8-normal   | True    | 21    | 54         |
| doorkey-8x8-shortcut | True    | 8     | 19         |
| Overall              | 7/7     | –     | –          |

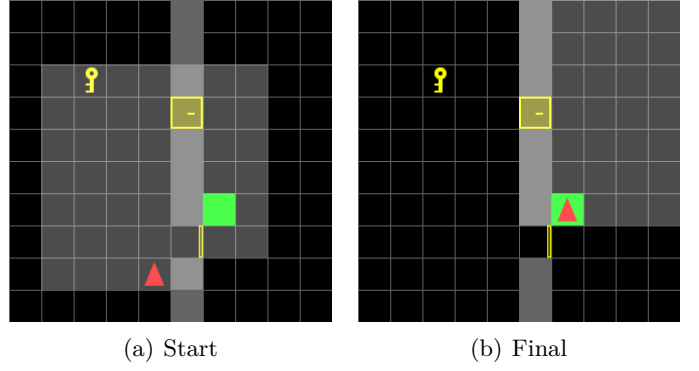


Fig. 9: Random map DoorKey-10x10-11.

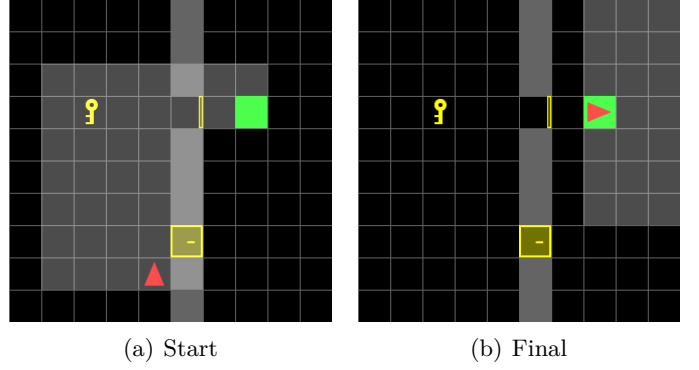


Fig. 10: Random map DoorKey-10x10-18.

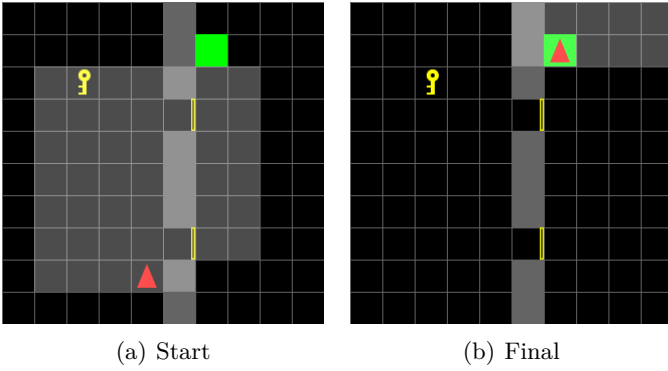


Fig. 7: Random map DoorKey-10x10-1: optimal trajectory visualization.

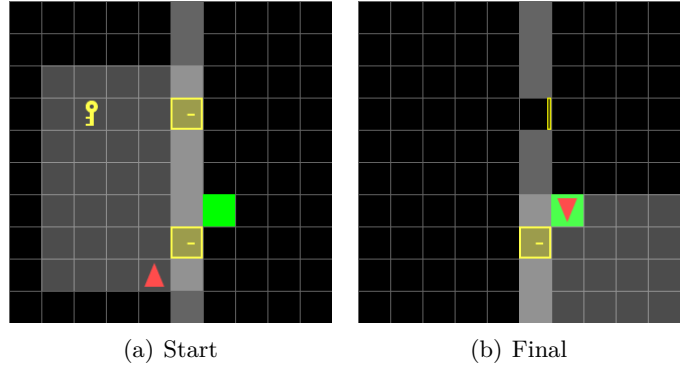


Fig. 11: Random map DoorKey-10x10-24.

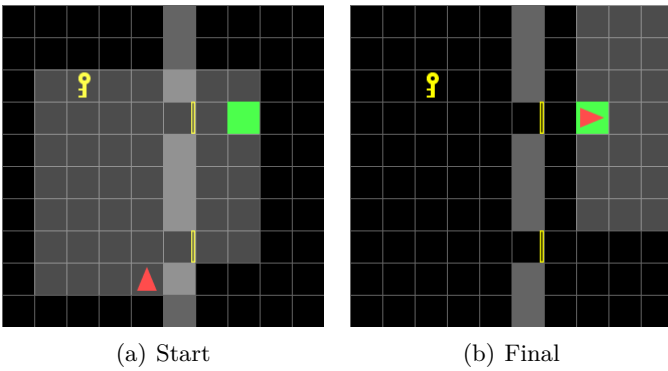


Fig. 8: Random map DoorKey-10x10-5.

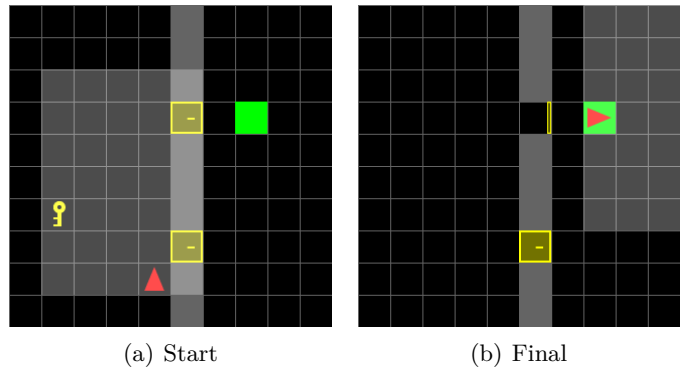


Fig. 12: Random map DoorKey-10x10-32.

TABLE III: Random-map trajectories: initial  $x_0 = (p_x^0, p_y^0, d^0, h^0)$  and total cost of the optimal trajectory (part 1/2).

| Environment          | $p_x^0$ | $p_y^0$ | $d^0$ | $h^0$ | Steps | Total cost |
|----------------------|---------|---------|-------|-------|-------|------------|
| DoorKey-10x10-1.env  | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-10.env | 4       | 8       | 3     | 0     | 12    | 32         |
| DoorKey-10x10-11.env | 4       | 8       | 3     | 0     | 6     | 14         |
| DoorKey-10x10-12.env | 4       | 8       | 3     | 0     | 20    | 53         |
| DoorKey-10x10-13.env | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-14.env | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-15.env | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-16.env | 4       | 8       | 3     | 0     | 17    | 44         |
| DoorKey-10x10-17.env | 4       | 8       | 3     | 0     | 9     | 25         |
| DoorKey-10x10-18.env | 4       | 8       | 3     | 0     | 9     | 25         |
| DoorKey-10x10-19.env | 4       | 8       | 3     | 0     | 10    | 26         |
| DoorKey-10x10-2.env  | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-20.env | 4       | 8       | 3     | 0     | 15    | 40         |
| DoorKey-10x10-21.env | 4       | 8       | 3     | 0     | 6     | 14         |
| DoorKey-10x10-22.env | 4       | 8       | 3     | 0     | 12    | 32         |
| DoorKey-10x10-23.env | 4       | 8       | 3     | 0     | 6     | 14         |
| DoorKey-10x10-24.env | 4       | 8       | 3     | 0     | 18    | 47         |
| DoorKey-10x10-25.env | 4       | 8       | 3     | 0     | 11    | 29         |

TABLE IV: Random-map trajectories (continued):  $x_0$  and total cost.

| Environment          | $p_x^0$ | $p_y^0$ | $d^0$ | $h^0$ | Steps | Total cost |
|----------------------|---------|---------|-------|-------|-------|------------|
| DoorKey-10x10-26.env | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-27.env | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-28.env | 4       | 8       | 3     | 0     | 19    | 50         |
| DoorKey-10x10-29.env | 4       | 8       | 3     | 0     | 9     | 25         |
| DoorKey-10x10-3.env  | 4       | 8       | 3     | 0     | 11    | 29         |
| DoorKey-10x10-30.env | 4       | 8       | 3     | 0     | 9     | 25         |
| DoorKey-10x10-31.env | 4       | 8       | 3     | 0     | 10    | 26         |
| DoorKey-10x10-32.env | 4       | 8       | 3     | 0     | 17    | 46         |
| DoorKey-10x10-33.env | 4       | 8       | 3     | 0     | 6     | 14         |
| DoorKey-10x10-34.env | 4       | 8       | 3     | 0     | 12    | 32         |
| DoorKey-10x10-35.env | 4       | 8       | 3     | 0     | 6     | 14         |
| DoorKey-10x10-36.env | 4       | 8       | 3     | 0     | 16    | 41         |
| DoorKey-10x10-4.env  | 4       | 8       | 3     | 0     | 19    | 50         |
| DoorKey-10x10-5.env  | 4       | 8       | 3     | 0     | 9     | 25         |
| DoorKey-10x10-6.env  | 4       | 8       | 3     | 0     | 9     | 25         |
| DoorKey-10x10-7.env  | 4       | 8       | 3     | 0     | 10    | 26         |
| DoorKey-10x10-8.env  | 4       | 8       | 3     | 0     | 17    | 46         |
| DoorKey-10x10-9.env  | 4       | 8       | 3     | 0     | 6     | 14         |

TABLE V: Known-Map Multi-Start/Multi-Orientation Results

| Environment          | Trials | Success Rate | Avg Cost |
|----------------------|--------|--------------|----------|
| doorkey-5x5-normal   | 8      | 1.00         | 28.50    |
| doorkey-6x6-direct   | 8      | 1.00         | 12.50    |
| doorkey-6x6-normal   | 8      | 1.00         | 31.50    |
| doorkey-6x6-shortcut | 8      | 1.00         | 18.00    |
| doorkey-8x8-direct   | 8      | 1.00         | 19.50    |
| doorkey-8x8-normal   | 8      | 1.00         | 53.00    |
| doorkey-8x8-shortcut | 8      | 1.00         | 17.00    |

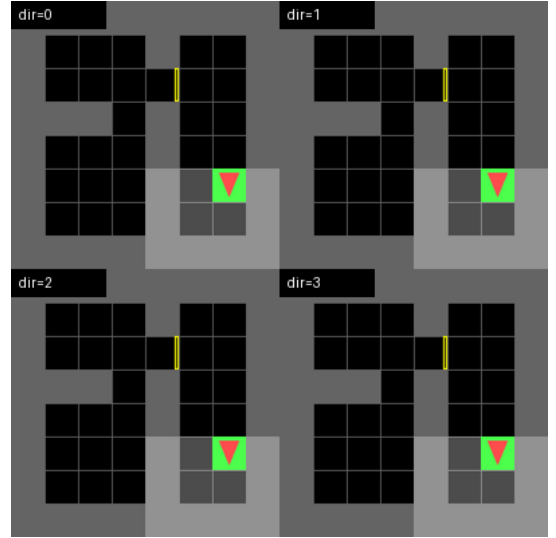


Fig. 13: Final states for doorkey-8x8-normal under four different initial orientations.